

Elastic Net Regression

Contents

1. Overview	2
2. The loss function	2
3. alpha and l1_ratio	3
4. The shape of the penalty	3
5. Correlated inputs: the grouping effect	4
6. Elastic Net on the diabetes data	5
7. When to use which	6
8. Summary	7
9. Key terms	7

Where this fits: Pipeline map step 8 (Model). See the [Course map](#).



- **Builds on:** Multicollinearity ([Note 27](#)); Regularisation ([Note 63](#)); Ridge regression ([Note 66](#)); Lasso regression ([Note 68](#)).

1. Overview

Key point: Elastic Net adds both the Ridge penalty and the Lasso penalty to the loss. One setting controls the overall strength, another the mix between the two.

The last Notes covered two regularised versions of linear regression:

- **Ridge** (L2): shrinks all coefficients, never to exactly 0. Suited to data where every input matters.
- **Lasso** (L1): can set coefficients to exactly 0, removing inputs. Suited to data where only some inputs matter.

With a large dataset of many columns, it is often impossible to know in advance which of the two fits better. **Elastic Net regression** uses both penalties at once and lets their mix be tuned like any other hyperparameter.

2. The loss function

Key point: Loss = squared error + $a \times$ (sum of squared coefficients) + $b \times$ (sum of absolute coefficients).

The Elastic Net loss is the squared error with both penalties added:

$$L = \sum_{i=1}^n (y_i - \hat{y}_i)^2 + a \sum_{j=1}^m \beta_j^2 + b \sum_{j=1}^m |\beta_j|$$

a sets the strength of the Ridge part and b the strength of the Lasso part. They are separate numbers, so the two parts can be weighted differently.

- With $b = 0$, it is Ridge.
- With $a = 0$, it is Lasso.
- With $a = b = 0$, it is plain linear regression.

3. alpha and l1_ratio

Key point: scikit-learn uses `alpha` for the total strength and `l1_ratio` for the share of the Lasso (L1) part.

Instead of a and b , scikit-learn's `ElasticNet` takes two other hyperparameters:

$$\text{alpha} = a + b \quad \text{l1_ratio} = \frac{b}{a + b}$$

In words: `alpha` is the total penalty, and `l1_ratio` is the fraction of it that goes to the L1 (Lasso) part. The rest, $1 - \text{l1_ratio}$, goes to the L2 (Ridge) part.

l1_ratio	Mix	Behaves like
0	all L2	Ridge
0.5 (default)	half and half	an even mix
0.9	90% L1, 10% L2	mostly Lasso
1	all L1	Lasso

With numbers: `alpha` 1 and `l1_ratio` 0.5 give $a = 0.5$ and $b = 0.5$. Going back is easy too: $b = \text{alpha} \times \text{l1_ratio}$ and $a = \text{alpha} - b$.

Extra: A common slip is to read `l1_ratio` 0.9 as “90% Ridge”. The name says what it measures: the L1 share. So 0.9 means 90% Lasso.

Extra: As with `Lasso`, scikit-learn scales the terms a little differently: it minimises $\frac{1}{2n} \sum (y_i - \hat{y}_i)^2 + \text{alpha} \cdot \text{l1_ratio} \sum |\beta_j| + \frac{1}{2} \text{alpha} (1 - \text{l1_ratio}) \sum \beta_j^2$. The idea is the same; only the scale of `alpha` differs.

4. The shape of the penalty

Key point: The Elastic Net penalty is between Ridge's circle and Lasso's diamond: rounded, but still with corners on the axes, so it can still set coefficients to 0.

For two coefficients, Figure 1 draws all the points where each penalty equals 1.

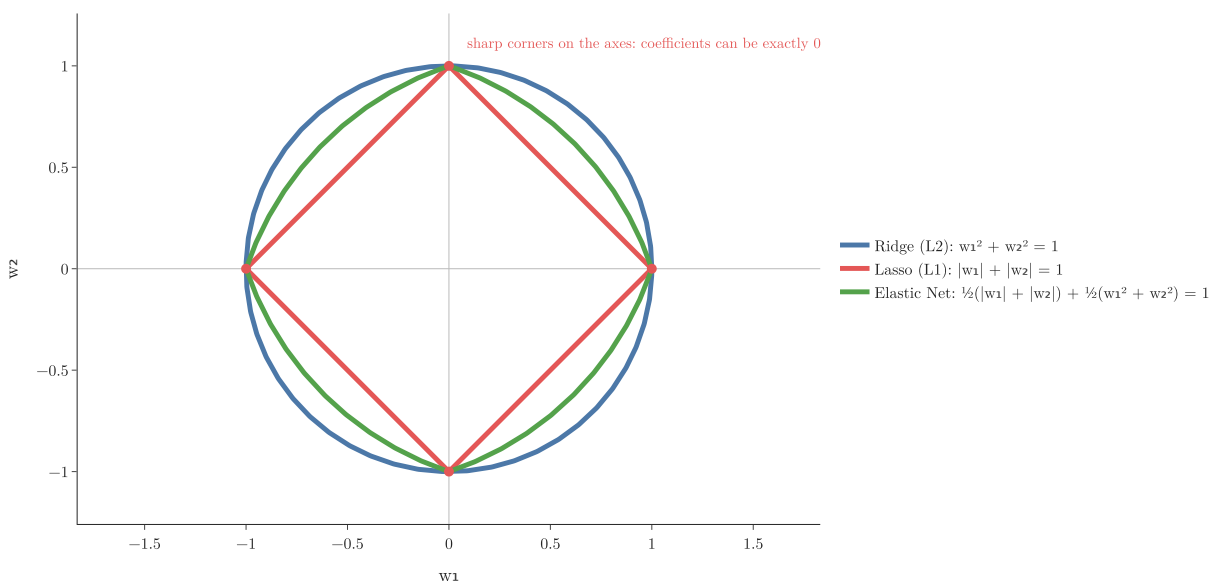


Figure 1: The penalty shapes of Ridge, Lasso and Elastic Net

- **Ridge** gives a circle: smooth everywhere.
- **Lasso** gives a diamond with sharp corners on the axes, where one coefficient is 0. Those corners are why Lasso answers often land exactly on 0 (the Lasso sparsity Note).
- **Elastic Net** is between the two. Its sides bulge outwards like the circle, but it keeps the corners, so it can still produce exact zeros.

5. Correlated inputs: the grouping effect

Key point: When inputs are strongly correlated, Lasso keeps one or two and drops the rest at random. Elastic Net shares the weight among them and still drops the useless inputs.

Elastic Net is especially recommended when inputs are strongly correlated with each other: **multicollinearity** (the regression assumptions Note). Height and weight are a typical pair: when one rises, the other usually does too.

Figure 2 uses 200 rows with six inputs. x_1 , x_2 and x_3 are three almost identical copies of one signal, and the output is 3 times that signal plus noise. The other three inputs are pure noise.

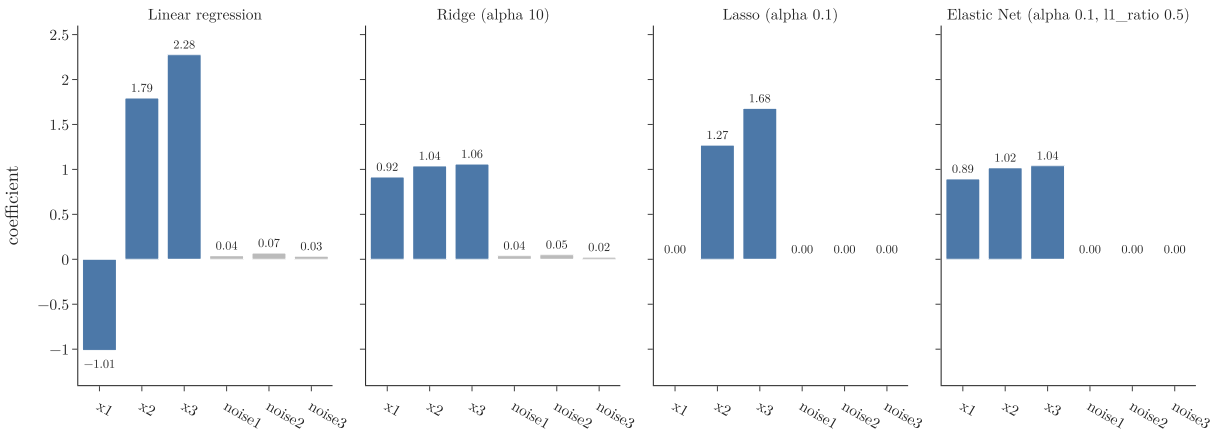


Figure 2: Coefficients of four models on three correlated inputs and three noise inputs

Model	x_1	x_2	x_3	Noise inputs
Linear regression	-1.01	1.79	2.28	about 0.05
Ridge (alpha 10)	0.92	1.04	1.06	about 0.04
Lasso (alpha 0.1)	0	1.27	1.68	0
Elastic Net (alpha 0.1, l1_ratio 0.5)	0.89	1.02	1.04	0

- **Linear regression** gives the three copies wild, unstable values; x_1 even gets a negative sign. Only their sum, about 3, is meaningful.
- **Ridge** shares the weight evenly, about 1 each, but keeps small coefficients on the noise inputs.
- **Lasso** drops the noise, but also drops x_1 , which is just as useful as the others. Which copy it drops is close to arbitrary.
- **Elastic Net** does both: it shares the weight evenly among the three copies, like Ridge, and sets the noise inputs to 0, like Lasso.

This sharing of weight among correlated inputs is called the **grouping effect**.

6. Elastic Net on the diabetes data

Key point: On the small diabetes data all four models score about the same; tune alpha and l1_ratio with cross-validation, not the test set.

Python: Elastic Net in scikit-learn.

```
from sklearn.linear_model import ElasticNet

enet = ElasticNet(alpha=0.005, l1_ratio=0.9)
enet.fit(X_train, y_train)
enet.score(X_test, y_test)          #  $R^2$  0.453
```

On the diabetes split used before (test size 0.2, random state 2):

Model	Test R^2
Linear regression	0.440
Ridge(alpha=0.1)	0.452
Lasso(alpha=0.01)	0.441
ElasticNet(alpha=0.005, l1_ratio=0.9)	0.453

Elastic Net scores slightly best here, but the differences are tiny. Also, these settings were found by trying values against this same test set, which flatters them.

The fair way is to tune both hyperparameters with cross-validation on the training data. `ElasticNetCV` does this; on this data it chose alpha 0.06 with l1_ratio 1 (pure Lasso), scoring 0.438 on the test set. Grid search and randomised search (later Notes) do the same job for any model.

Python: Tuning with cross-validation.

```
from sklearn.linear_model import ElasticNetCV

cv = ElasticNetCV(l1_ratio=[0.1, 0.5, 0.7, 0.9, 0.95, 0.99, 1],
                  alphas=np.logspace(-4, 0, 50), cv=5)
cv.fit(X_train, y_train)
cv.alpha_, cv.l1_ratio_           # 0.06, 1.0
```

Extra: `SGDRegressor(penalty="elasticnet", alpha=..., l1_ratio=...)` trains the same kind of model with stochastic gradient descent (the SGD Note). `ElasticNet` is built specifically for this problem and is usually the better choice; SGD helps when the data is too large for memory.

7. When to use which

Key point: Ridge when all inputs matter, Lasso when only some do, Elastic Net when unsure or when inputs are strongly correlated.

Situation	Choice
Most inputs are useful	Ridge
Only some inputs are useful; want feature selection	Lasso
Many columns, unclear which kind	Elastic Net
Strongly correlated inputs	Elastic Net (or Ridge)

In practice, Elastic Net with l1_ratio tuned by cross-validation covers all three cases: the search can land on Ridge-like or Lasso-like settings if those fit best.

8. Summary

- Elastic Net loss = squared error + L2 penalty + L1 penalty.
- `alpha` sets the total strength; `l1_ratio` sets the L1 (Lasso) share: 0 is Ridge, 1 is Lasso.
- It can still set coefficients to 0, but shares weight among correlated inputs (the grouping effect).
- Tune `alpha` and `l1_ratio` with cross-validation, for example with `ElasticNetCV`.

9. Key terms

Term	Meaning
Elastic Net regression	Linear regression with both the L1 and the L2 penalty
<code>l1_ratio</code>	The share of the total penalty given to the L1 (Lasso) part
Grouping effect	Elastic Net's tendency to give correlated inputs similar coefficients instead of keeping only one
<code>ElasticNetCV</code>	scikit-learn's Elastic Net that picks <code>alpha</code> and <code>l1_ratio</code> by cross-validation